Mobile Robotics Lab - Final Report

1. Lab Objective

The objective of this final lab session was to implement image-based perception and control using the TurtleBot3 Waffle's onboard camera. The goal was to establish a complete perception-to-control pipeline: subscribing to camera topics, processing images using OpenCV to isolate specific colored objects, computing positional errors, and generating real-time cmd_vel motion commands to align and navigate toward the targets.

2. Implementation & Task Completion

The provided requirements (Tasks 1 through 5) and suggested exercises were successfully integrated into a single, cohesive ROS 2 node (camera_follower.py).

2.1. Image Processing Pipeline (Tasks 1 & 2)

- **Subscription:** The node subscribes to /camera/image_raw.
- **Conversion:** Using CvBridge, the ROS Image message (RGB) is successfully converted into an OpenCV matrix (bgr8).
- **Segmentation:** The image is converted from BGR to HSV color space to decouple color (Hue) from lighting (Value/Saturation).

2.2. Feature Extraction & Centroid Computation (Task 3)

Using OpenCV's cv2.moments(), the algorithm calculates the center of mass (centroid) of the segmented binary masks. A minimum area filter (area > 500) was implemented to ignore random background noise and prevent the robot from tracking false positives.

2.3. Proportional Control & Navigation (Tasks 4 & 5)

A proportional controller (P-controller) was designed to minimize the visual error (the difference between the image's horizontal center and the object's centroid).

- **Alignment:** The robot applies an angular velocity (angular.z) proportional to the pixel error.
- **Forward Motion:** Once the error falls below the center_threshold (40 pixels), the robot sets linear.x to 0.15 m/s to move forward.
- **Proximity Stop:** By calculating the physical pixel area of the object, the robot estimates its distance. Once the area exceeds a predefined threshold (550,000 pixels), it halts all motion.

2.4. Advanced State Machine (Exercises 3 & 4)

To address the advanced exercises, the node was upgraded to utilize a **Finite State Machine (FSM)**. Instead of a single behavior, the robot sequentially hunts multiple targets:

1. SEARCH_RED: Rotates in place searching for a red sphere.
2. TRACK_RED: Aligns and drives toward the red sphere until the stop area is reached.
3. SEARCH_GREEN: Abandons the red target, rotates, and searches for a green cuboid.
4. TRACK_GREEN: Aligns and drives toward the green cuboid.
5. DONE: Stops tracking and spins indefinitely to signal the end of the demonstration.

3. Observations & Controller Tuning (Deliverable 6)

- **Tuning Proportional Gain (K_p):** Experimentation showed that a high K_p caused violent oscillations—the robot would overshoot the center of the object and endlessly zig-zag. A final tuned value of $K_p = 0.001$ provided smooth, steady rotation. A maximum angular velocity clamp ([-1.5, 1.5]) was added to prevent dangerous spin speeds when the error was extremely large.
- **Color Segmentation Challenges (The Red Hue Wrap-around):**
  A significant challenge in HSV processing is that the color Red exists at both the bottom (0-10) and the top (160-180) of the Hue spectrum. A single threshold failed to capture the whole sphere reliably. This was solved by creating two separate masks (mask1 and mask2) and combining them using cv2.bitwise_or.
- **Environmental Noise:** Initially, large background objects (like Gazebo's green walls) caused the centroid to shift erratically. This was resolved by placing distinct geometric shapes (a red sphere and a green cuboid) and tuning the bounding box thresholds to track isolated objects rather than background geometry.

4. Deliverables Checklist References

- **1. GitHub Repository:** *[Insert Your GitHub Link Here]*
- **3. Screenshots:** *[Insert/Attach Screenshots of your CV2 mask windows and Camera View here]*
- **4. RQT Graph:** *[Insert/Attach your rqt_graph screenshot here]*
- **5. Demonstration Video:** *[Insert video link or note video file submission here]*

5. Source Code (Deliverable 2)

In the package folder

6. Conclusion (Deliverable 7)

This lab successfully demonstrated the integration of computer vision into robotic navigation. By taking raw sensory data, isolating specific geometric and chromatic features using OpenCV, and feeding those features into a dynamic P-controller, the TurtleBot was granted autonomous tracking capabilities. Developing the state-machine logic also provided hands-on experience with complex behavior trees, showcasing how perception and closed-loop control work together in modern mobile robotics.